You don't need to revise everything.

You need to revise the concepts that are most likely to come up and understand them well enough to explain them clearly.

Here's my 7-day technical interview revision plan:

💻 **Day 1**: **DSA Patterns** • Sliding Window • Two Pointers • Hashing • Binary Search • Stack & Queue • DFS / BFS

☕ **Day 2**: **OOP** & **Core Concepts** • Encapsulation • Inheritance • Polymorphism • Abstraction • SOLID principles • Exception Handling

🗄️ **Day 3**: **SQL** & **Databases** • JOINs • GROUP BY & HAVING • Subqueries • Indexing • Transactions • Window Functions • Query Optimization

🔧 **Day 4**: **APIs** & **Backend** • REST APIs • HTTP Methods • Status Codes • Authentication & Authorization • JWT / OAuth • Pagination • Rate Limiting • API Error Handling

⚡ **Day 5**: **System Design** • Caching • Load Balancing • Database Scaling • Replication • Queues • Horizontal vs Vertical Scaling • CAP Theorem • Design one complete system

🌐 **Day 6**: **OS**, **Networking** & **Security** • Processes vs Threads • Concurrency • Deadlocks • Memory Management • HTTP vs HTTPS • DNS • TCP vs UDP • Cookies vs Sessions • CORS • SQL Injection • Encryption vs Hashing

🚀 **Day 7**: **Git**, **Testing** & **Mock Interview** • Git Branching • Merge vs Rebase • Conflict Resolution • Revert vs Reset • Pull Requests • Unit & API Testing • Debugging • Logging • Edge Cases • 1–2 Mock Interviews

If You Have Even Less Time:

Focus on these first:

1. Sliding Window
2. Two Pointers
3. Hashing
4. Binary Search
5. DFS/BFS
6. OOP Principles
7. SQL JOINs
8. Indexing
9. REST APIs
10. Authentication
11. Caching
12. Load Balancing
13. HTTP/HTTPS
14. Processes vs Threads
15. Git Merge vs Rebase

And don't forget:

Know Your Projects.

Be ready to explain: • What you built • Why you chose the technology • Your architecture • Challenges you faced • How you solved them • Performance improvements • Trade-offs

You don’t need to know everything.

You need to know the fundamentals well and be able to explain your thinking.